

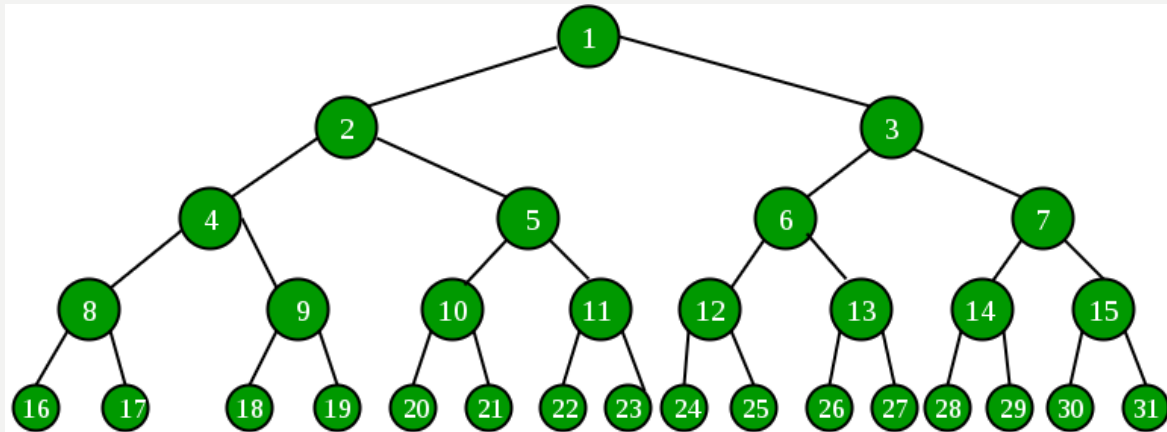
ESTRUTURA DE DADOS

ÁRVORE BINÁRIA

ÁRVORES BINÁRIAS

INTRODUÇÃO

- Uma árvore binária é uma estrutura de dados mais geral que uma [lista encadeada](#)
- Abordaremos algumas operações básicas sobre árvores binárias.



ÁRVORES BINÁRIAS

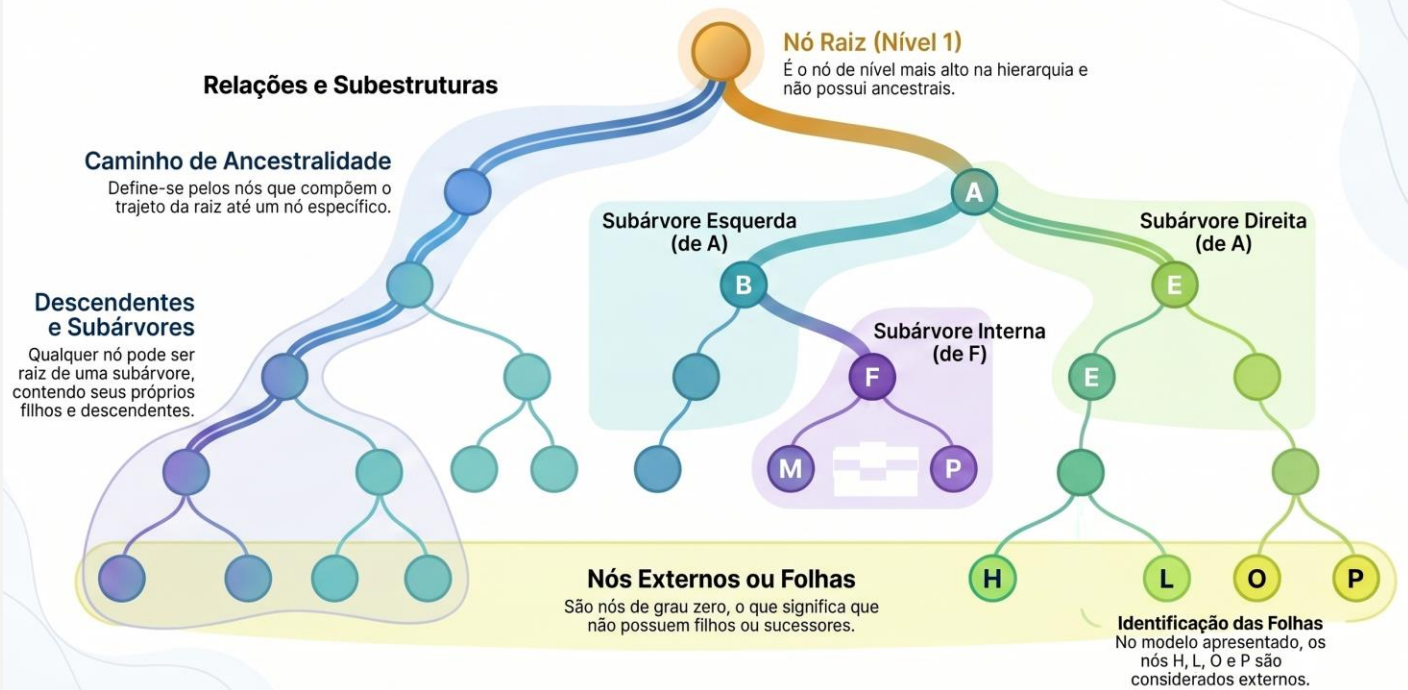
ÁRVORES

- Em computação , uma árvore é um modelo *abstrato* de uma estrutura hierárquica, não linear, construída composta por *nós* com relação de parentesco (pais e filhos). A forma mais natural definirmos uma estrutura de árvore é utilizando a recursividade.
- Uma árvore é composta por um conjunto finito de nós
- Sua composição : nó *r* (raiz), que contém zeros ou mais sub-árvores. Estas são filha de *r*.
- Nós com filhos são chamados de *nós internos*
- Nós que não contém filhos, são chamados de *nós externos* (*folhas*)

ÁRVORES BINÁRIAS

Anatomia de uma Árvore Binária: Estrutura e Hierarquia

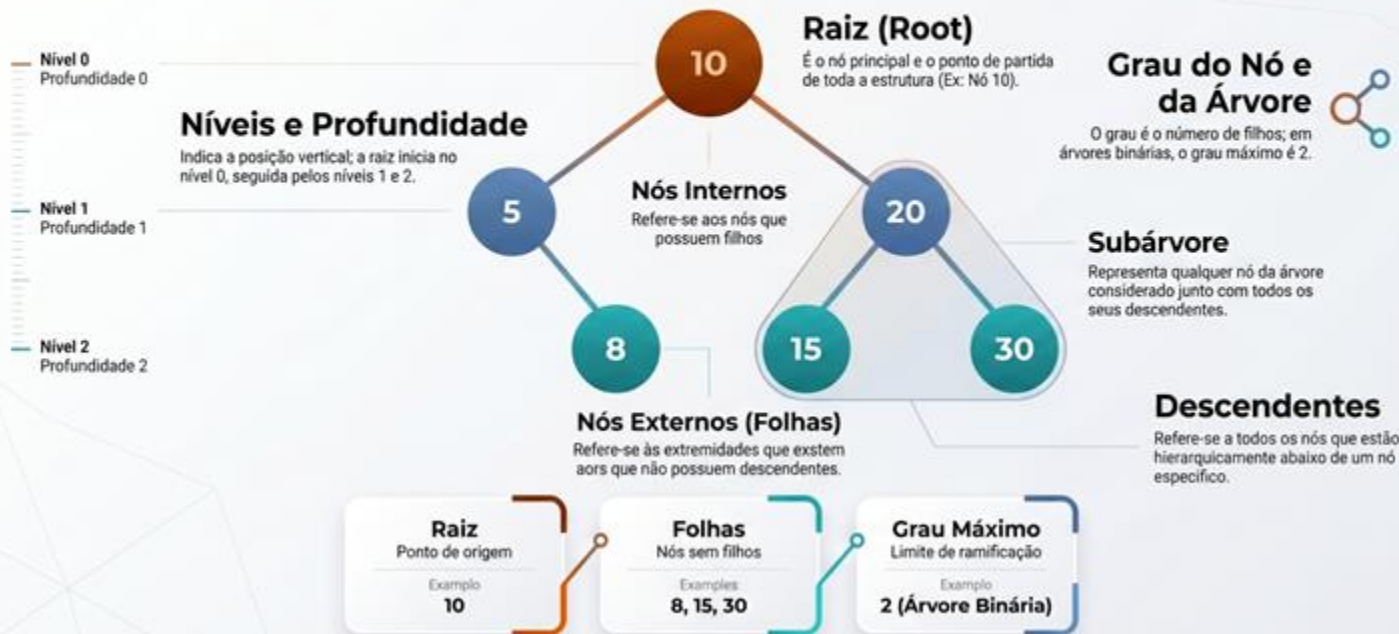
Este guia define os componentes essenciais de uma árvore binária, desde a raiz até as extremidades (folhas). Ele detalha como os nós se organizam em níveis e como as relações de ancestralidade e subestruturas definem a hierarquia do sistema.



ÁRVORES BINÁRIAS

Desvendando Árvores Binárias: Estrutura e Terminologia

Uma árvore binária é uma estrutura de dados fundamental onde cada elemento (nó) serve como base para, no máximo, dois outros. Este guia define os termos técnicos essenciais para identificar posições e relações dentro dessa hierarquia.



ÁRVORES BINÁRIAS

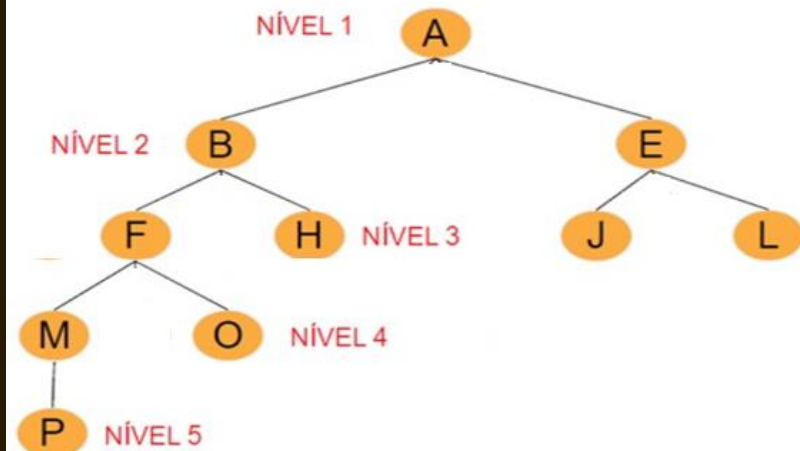
TERMINOLOGIA

Com base na estrutura da árvore fornecida, aqui está o detalhamento técnico da profundidade e altura de cada nó.

- **Profundidade:** É a distância do nó até a **raiz** (contando as arestas). A raiz sempre tem profundidade 0.
- **Altura:** É a distância do nó até a **folha mais profunda** abaixo dele. Um nó folha sempre tem altura 0.

ÁRVORES BINÁRIAS

TERMINOLOGIA



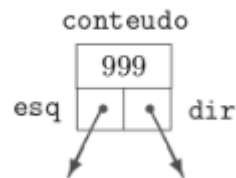
Nó	Nível	Profundidade	Altura
A	1	0	4
B	2	1	3
E	2	1	1
F	3	2	2
H	3	2	0
J	3	2	0
L	3	2	0
M	4	3	1
O	4	3	0
P	5	4	0

ÁRVORES BINÁRIAS

ÁRVORES BINÁRIAS -NÓS E FILHOS

- Uma árvore binária (= *binary tree*) é um conjunto de registros que satisfaz certas condições. As condições não serão dadas explicitamente, mas elas ficarão implicitamente claras no contexto. Os registros serão chamados *nós* (poderiam também ser chamados *células*). Cada nó tem um endereço. Suporemos por enquanto que cada nó tem apenas três campos: um número inteiro e dois ponteiros para nós. Os nós podem, então, ser definidos assim:

```
typedef struct reg {  
    int  conteudo; // conteúdo  
    noh *esq;  
    noh *dir;  
} noh; // nó
```



ÁRVORES BINÁRIAS

NÓS E FILHOS

- O campo conteúdo é a “*carga útil*” do nó; os dois outros campos servem apenas para dar estrutura à árvore. O campo **esq** de cada nó contém NULL ou o endereço de outro nó. Uma hipótese análoga vale para o campo **dir**.



ÁRVORES BINÁRIAS

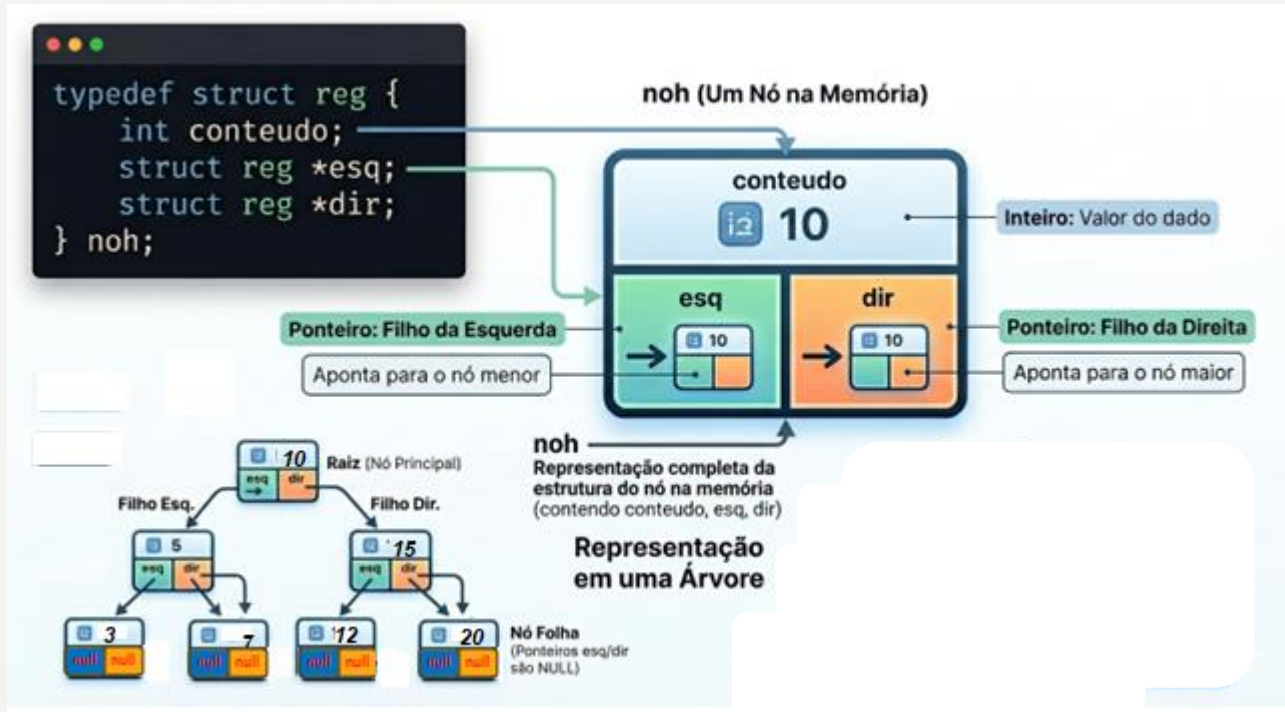
NÓS E FILHOS

- Se o campo `esq` de um nó `P` é o endereço de um nó `E`, diremos que `E` é o filho esquerdo de `P`. Analogamente, se `P.dir` é igual a `D`, diremos que `D` é o filho direito de `P`. Se um nó `F` é filho (esquerdo ou direito) de `P`, diremos que `P` é o pai de `F`. Uma folha (= *leaf*) é um nó que não tem filho algum.

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C



ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

Para implementar uma árvore como a da imagem em **C**, precisamos transpor essa estrutura visual para a memória do computador. Em C, isso é feito através de **ponteiros** e **structs**, onde cada nó é um objeto independente que conhece apenas o endereço de seus "filhos".

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

A Estrutura do Nó (Struct)

Antes de pensar em funções, definimos o que é um "nó". No caso de uma árvore binária (como a da imagem, onde cada pai tem no máximo dois filhos), a estrutura contém o dado e dois ponteiros:

```
typedef struct No {  
    char dado;           // Armazena 'A', 'B', etc.  
    struct No *esquerda; // Ponteiro para o filho da esquerda  
    struct No *direita;  // Ponteiro para o filho da direita  
} No;
```

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

A Recursividade

A árvore é, por definição, uma **estrutura recursiva**: uma árvore é composta por uma raiz e subárvores, que por sua vez também são árvores. Por isso, funções recursivas são a forma mais natural e elegante de manipular esses dados.

Por que usar recursividade?

- **Simplicidade:** Resolver um problema na árvore inteira resume-se a resolvê-lo para o nó atual e "chamar a função novamente" para os filhos.
- **Navegação:** Para chegar ao nó **P** (nível 5), a função entra em si mesma sucessivamente até encontrar um ponteiro `NULL`.

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

Exemplo: Calculando a Altura

Para descobrir a altura de um nó, a lógica recursiva funciona assim:

"A altura deste nó é 1 + a maior altura entre meus dois filhos".

```
int calcularAltura(No *raiz) {  
    if (raiz == NULL) return -1; // Caso base: nó inexistente não tem altura  
  
    int altEsq = calcularAltura(raiz->esquerda);  
    int altDir = calcularAltura(raiz->direita);  
  
    if (altEsq > altDir)  
        return altEsq + 1;  
    else  
        return altDir + 1;  
}
```

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

Percursos (Visitando os Nós)

Para listar os nós da imagem, usamos diferentes estratégias recursivas dependendo da ordem desejada:

- **Pré-ordem (Raiz, Esq, Dir):** Visita **A**, depois mergulha na subárvore de **B**.
- **Em-ordem (Esq, Raiz, Dir):** Muito usada para manter dados ordenados.
- **Pós-ordem (Esq, Dir, Raiz):** Ideal para **deletar** a árvore da memória, pois você apaga os filhos antes de apagar o pai.

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

Exemplo de Percurso Pré-ordem:

```
void imprimirArvore(No *raiz) {  
    if (raiz != NULL) {  
        printf("%c ", raiz->dado);    // Visita a Raiz  
        imprimirArvore(raiz->esquerda); // Recursão para a Esquerda  
        imprimirArvore(raiz->direita);  // Recursão para a Direita  
    }  
}
```

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

Gestão de Memória

Em C, como os nós são criados dinamicamente com `malloc()`, a recursividade também é essencial para liberar a memória. Uma função recursiva percorre cada "galho" até as folhas (como o nó **P**, **O** ou **L**) e vai liberando os espaços de volta até a raiz **A**, garantindo que nenhum ponteiro fique perdido (memory leak).

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM C

Implementar árvores em C sem recursividade exigiria o uso de estruturas auxiliares complexas (pilhas manuais). A recursividade aproveita a **pilha de execução do sistema** para rastrear onde estamos na árvore, tornando o código limpo e fiel à hierarquia visual que vemos no diagrama.